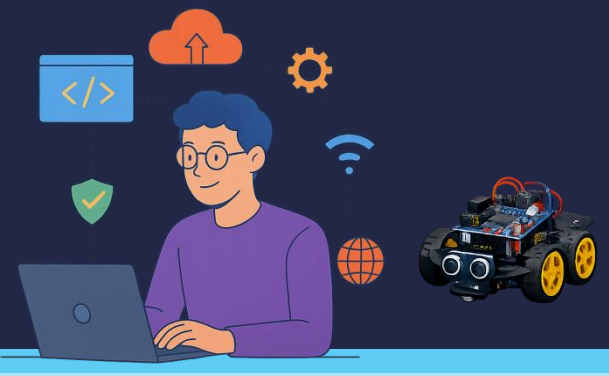
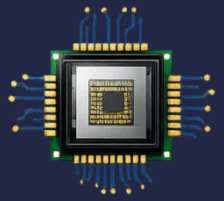
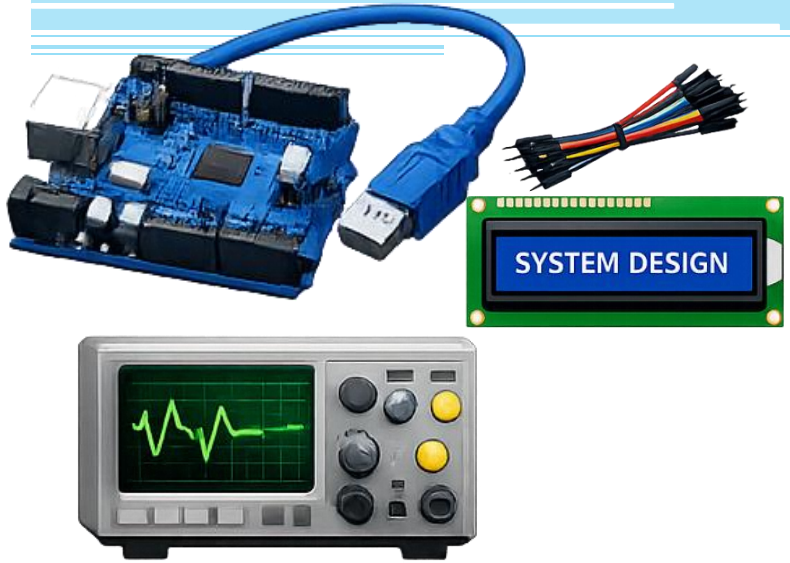


CS-301 Microprocessor-Based System Design



Lecture 9 and 10: Timers/Counters and PWM Concepts

Course Teacher: Syeda Ramish Fatima



ATmega162 – Timer/Counter0 Overview

- **Timer/Counter0** is an 8-bit timer module used for:
  - Time delay generation
  - Event counting
  - Frequency generation
  - PWM generation
- Key Features:
  - 8-bit counter (0 → 255)
  - Selectable prescaler
  - Compare Match capability
  - Multiple operating modes:
    - Normal Mode
    - CTC Mode
    - Fast PWM
    - Phase Correct

What is Fast PWM?

Fast PWM (Pulse Width Modulation) is a timer mode where the timer counts up from 0 to a maximum value (TOP) as fast as possible and then instantly resets back to 0, repeating this cycle continuously.

During this counting, the timer output pin is toggled based on a compare match value, creating a PWM waveform — a square wave where the pulse width (the "on" time) is controlled by software

This means you can slow down the counting speed to extend the timer period.

The timer can be compared to a value stored in a compare register.

The timer simply counts from 0 to 255 repeatedly.

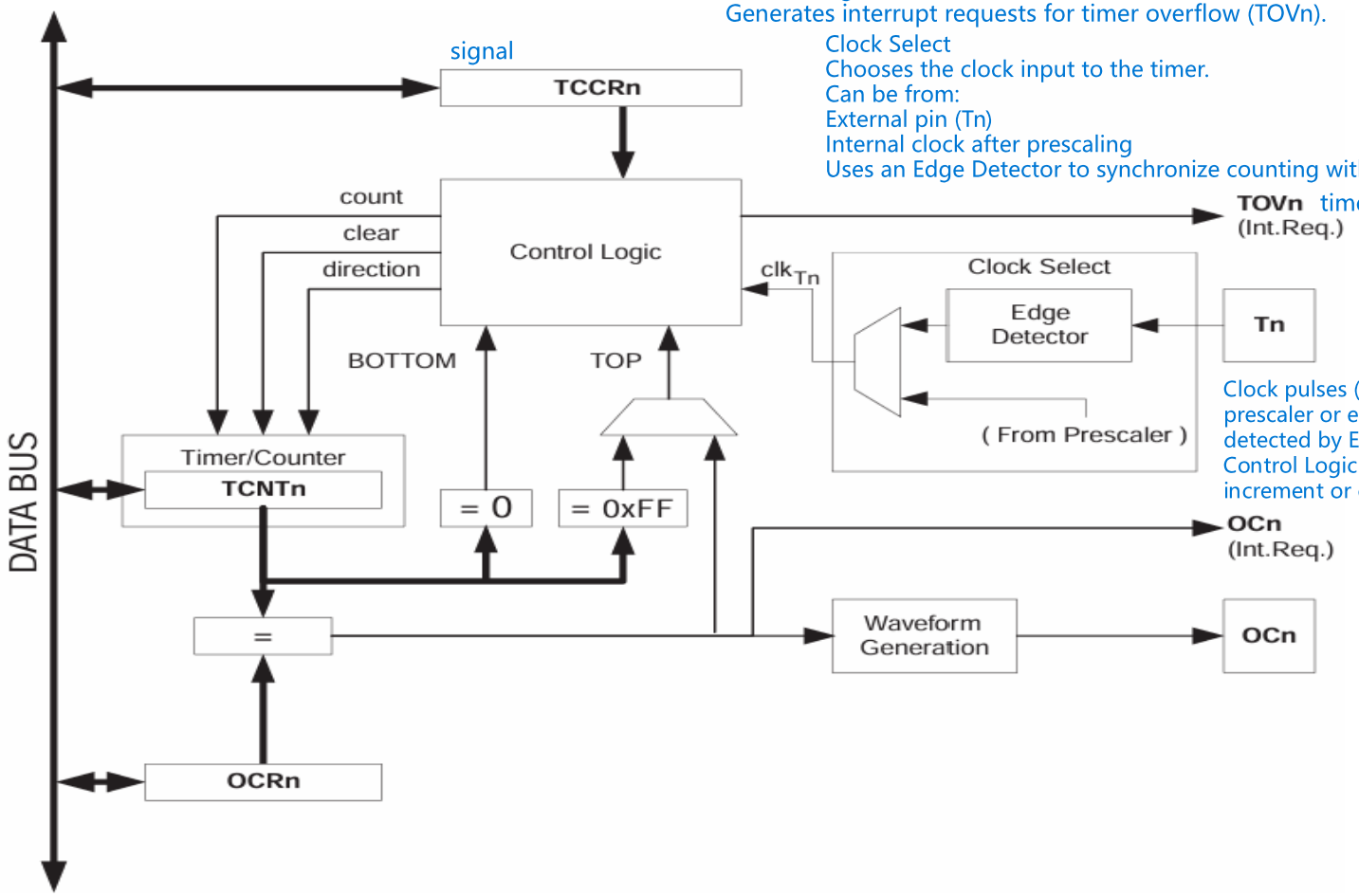
clear time on compare When it reaches compare value, it resets to 0 automatically.

PWM = Pulse Width Modulation (used to control brightness, speed, etc.)Output toggles based on compare match to create a PWM signal with varying duty cycle.

Phase Correct PWM is a timer mode where the timer counts up from 0 to TOP, then counts back down from TOP to 0, repeatedly — creating a symmetrical “up and down” counting pattern.

Intefacing of ATmega162 on-chip Timer/Counters

Figure 33. 8-bit Timer/Counter Block Diagram



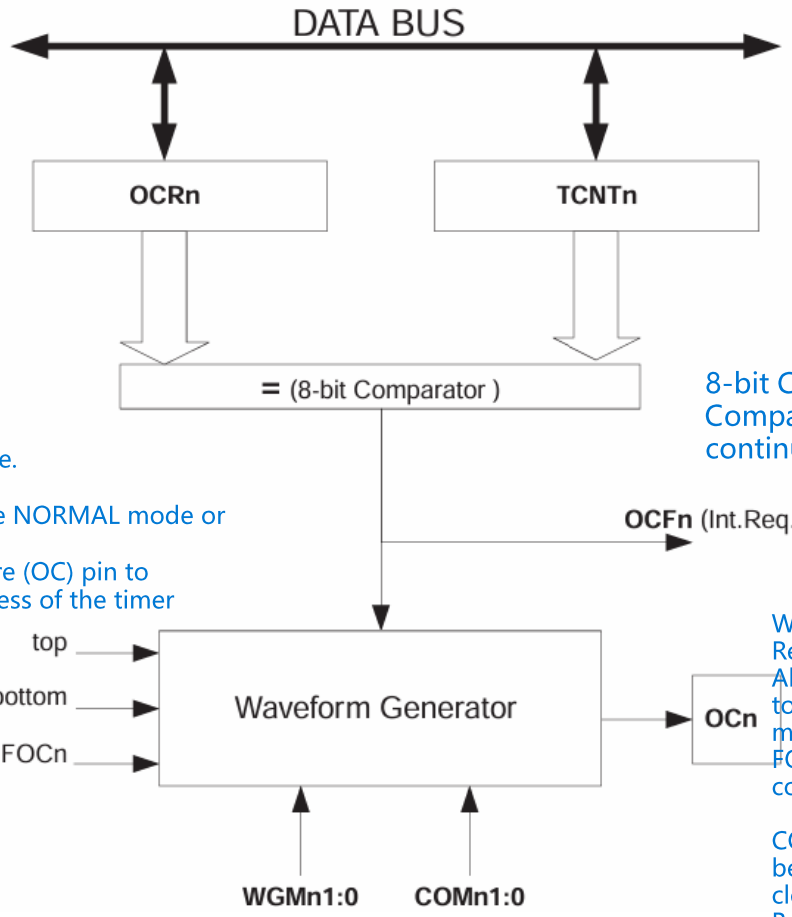
. Control Logic
controlling the timer operation.
Receives control signals from TCCRn and status from the timer.
Decides when to count, clear the timer, change counting direction.
Controls signals like BOTTOM and TOP conditions.
Generates interrupt requests for timer overflow (TOVn).

Clock Select
Chooses the clock input to the timer.
Can be from:
External pin (Tn)
Internal clock after prescaling
Uses an Edge Detector to synchronize counting with clock edges.

Clock pulses (from internal prescaler or external Tn pin) are detected by Edge Detector.
Control Logic uses these pulses to increment or decrement TCNTn.

Block Diagram Output Compare Unit

Figure 35. Output Compare Unit, Block Diagram



OCRn (Output Compare Register n)
Holds the compare value set by the program.
This is the target value the timer compares against.

8 bit counter, time counter register

Holds the current count of the 8-bit timer/counter.
This register increments (or decrements) on each clock tick (depending on mode).

8-bit Comparator (=)
Compares the value in OCRn with the value in TCNTn continuously.

FOC stands for Force Output Compare.

Present in non-PWM modes only (like NORMAL mode or CTC).
It is used to force the Output Compare (OC) pin to toggle/set/clear immediately, regardless of the timer counter value.

OCFn (Output Compare Flag n)
This flag is set when a compare match occurs.

OCn (Output Compare pin n)
The physical output pin that reflects the waveform or pulse generated.
Used to drive hardware signals such as PWM signals to control motors, LEDs, etc.

Waveform Generator
Receives input from the comparator
Also takes signals:
top and bottom: define limits for counting (e.g., max and min values, depending on mode).
FOCn (Force Output Compare): used to force an output compare action.

COMn1:0 (Compare Output Mode bits): control the behavior of the output pin on compare events (toggle, clear, set).
Based on these inputs, it generates waveforms or controls output pins accordingly.

WGMn1:0 (Waveform Generation Mode bits):
tells mode (normal, PWM, CTC, etc.).

ATmega162 – Timer/Counter0 Conceptual flow diagram

Clock → Prescaler → 8-bit Counter (TCNT0)

↓

Compare Unit (OCR0)

↓

Interrupt / OC0 pin

Important Registers

Register	Function
TCNT0	Timer counter value
OCR0	Output Compare Register
TCCR0	Timer/Counter control register
TIMSK	Interrupt enable
TIFR	Interrupt flag register

TIMSK (Timer/Counter Interrupt Mask Register) e.g., overflow, compare match)
Controls which timer interrupts are enabled (unmasked).

Indicates which timer interrupts have occurred
1 = event occurred, cleared by writing 1

Imortant Registers_TimerCounter0.pdf

TCCRn controls:
Clock source selection:
Which clock drives the timer (internal clock with prescaler, external clock pin, or no clock to stop the timer).
Prescaler settings:
Divides the system clock to slow down the timer counting speed (e.g., divide by 1, 8, 64, 256, 1024).
Counting mode:
Sets the timer's mode of operation, such as:
Normal counting mode
CTC (Clear Timer on Compare Match)
Fast PWM
Phase Correct PWM
Output Compare behavior:
Defines how the output pin (OCn) behaves on compare match events (toggle, clear, set).
Other control bits:
May control counting direction, force output compare, or enable/disable interrupts.

TIMSK stands for Timer/Counter Interrupt Mask Register.

It is used to enable or disable timer-related interrupts for Timer0, Timer1, and Timer2.
Each timer has its own interrupt sources, like overflow, compare match, and in some timers, input capture.
Setting a bit to 1 enables the interrupt; setting it to 0 disables it

While TIMSK is used to enable or disable interrupts, TIFR is used to check if a timer event has occurred.
It holds flags that indicate whether a specific timer event, like overflow or compare match, has happened.

Operation Modes

1. Normal Mode

Operation:

- Timer counts from 0 → 255
- On overflow (255 → 0), overflow flag is set
- Optional interrupt can be generated
- **WGM01 = 0 WGM00 = 0**

Applications:

- Time delay generation
- Time measurement
- Periodic interrupt generation
- Counts like: 0 → 1 → 2 → ... → 255 → Overflow → 0

What is Overflow Time?

The overflow time is the time it takes for the timer/counter register (TCNT) to count from 0 up to its maximum value and then roll over (overflow) back to 0.

Example (Normal Mode)

Clock = 16 MHz

Prescaler = 64

Overflow time:

T=1/976≈1.024ms

Overflow occurs every 1.024 ms

Overflow Frequency:

$$f_{overflow} = \frac{f_{clk}}{N \times 256}$$

$$f_{overflow} = \frac{16,000,000}{64 \times 256}$$

$$= 976Hz$$

Operation Modes

Basic Normal Mode Code:

```
TCCR0 = (1<<CS01) | (1<<CS00);   // Prescaler 64
TCNT0 = 0;                       // Start counting

while(!(TIFR & (1<<TOV0)));     // Wait for overflow
TIFR |= (1<<TOV0);              // Clear flag
```

TIFR = TIFR | x
It sets the bits in x to 1 without changing other bits

(TCCR0 = (1<<CS01) | (1<<CS00);
TCCR0 is the Timer/Counter Control Register for Timer0.
Bits CS02, CS01, CS00 control the timer clock prescaler.
(1<<CS01) | (1<<CS00) sets CS01 = 1 and CS00 = 1, so prescaler = 64.
This means the timer counts at (CPU clock / 64)

TOV0 – Bit position in TIFR that corresponds to Timer0 Overflow Flag.

1 << TOV0 – Bit shift operation

1 in binary: 00000001
Shift left by TOV0 positions → sets only that bit to 1.
Example: If TOV0 = 0 → 1 << 0 = 00000001

it means shifting the 1 in 0000001 to the times stored in tovo so that only tovo bit is changed

Operation Modes

2. CTC Mode (Clear Timer on Compare)

On every compare match:
OC0 pin toggles
This produces a 1 kHz square wave
use 2n forula

Operation:

- Timer counts from 0 → OCR0
- When TCNT0 = OCR0:
  - Compare match occurs
  - Counter resets to 0
  - Optional interrupt generated
- **WGM01 = 1 WGM00 = 0**

Applications

- Precise timing
- Custom frequency generation
- Square wave generation
- Better than normal mode for fixed delays

$$f_{CTC} = \frac{f_{clk}}{2 \times N \times (1 + OCR0)}$$

(When toggling OC0 pin)

If using interrupt only:

$$f = \frac{f_{clk}}{N \times (1 + OCR0)}$$

Example (CTC Mode Design)

Clock = 16 MHz

Prescaler = 64

Desired frequency = 1 kHz

Using interrupt method:

$$OCR0 = \frac{f_{clk}}{N \times f} - 1$$

$$OCR0 = \frac{16,000,000}{64 \times 1000} - 1$$

$$= 250 - 1$$

$$OCR0 = 249$$

Operation Modes

CTC Mode Code:

```
DDRB |= (1<<PB3);            // OC0 as output

TCCR0 |= (1<<WGM01);         // CTC mode
TCCR0 |= (1<<COM00);         // Toggle OC0 on compare
TCCR0 |= (1<<CS01) | (1<<CS00); // Prescaler 64

OCR0 = 249;                  // 500 Hz output
```

Pulse Width Modulation

- Pulse Width Modulation (PWM) is a technique used to control the average power delivered to a load by varying the width of digital pulses while keeping frequency constant.
- Output switches between HIGH and LOW.
- Frequency remains constant.
- Duty cycle is varied to control power.
- Widely used in digital systems to produce analog-like control.
- *Some Importants terms*
  - Period (T): Time for one complete cycle.
  - Frequency (f): f = 1/T
  - ON Time (Ton): Duration signal stays HIGH.
  - OFF Time (Toff): Duration signal stays LOW.
  - Duty Cycle (D): D = Ton/T × 100%

Ton = Time when signal is high

Ton = 3 T = totaltime = 10